FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering

Academic Year 2025-2026

Rubrics for Lab Experiments

Class: B.E. Computer Engineering **Subject Name:** SMA Lab

Semster: VIII **Subject Code:** CSDL8023

Practical No.	3
Title:	Data Cleaning and Storage- Preprocess, filter and store social media data for business
Date of Performance:	27-11-2025
Roll No.	9913
Student	Mark Lopes

Evaluation

Performance Indicator	Below Average	Average	Good	Excellent	Marks
On Time Submission (2)	Not submitted(0)	Submitted after deadline (1)	Early or on time submission (2)	—	
Test cases and output (4)	Incorrect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

Signature of the Teacher:

Aim: Data Cleaning and Storage- Preprocess, filter and store social media data for business

---

Code:

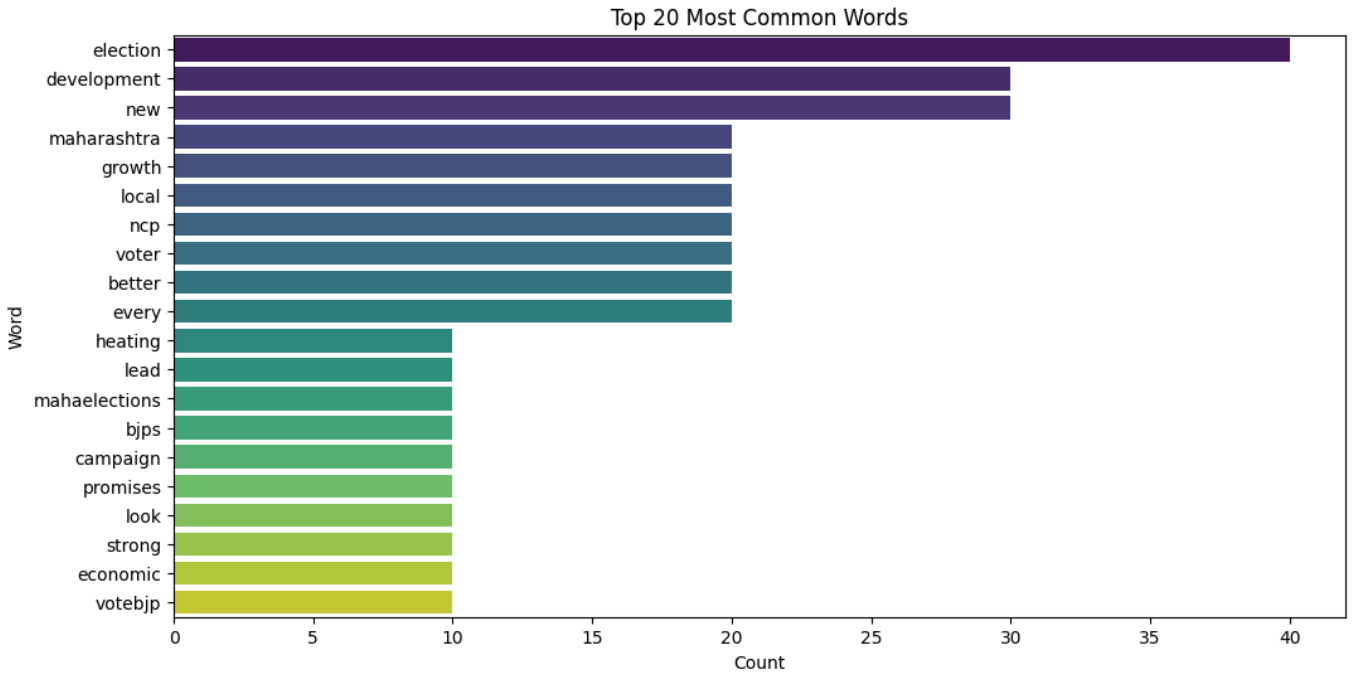
./main.py

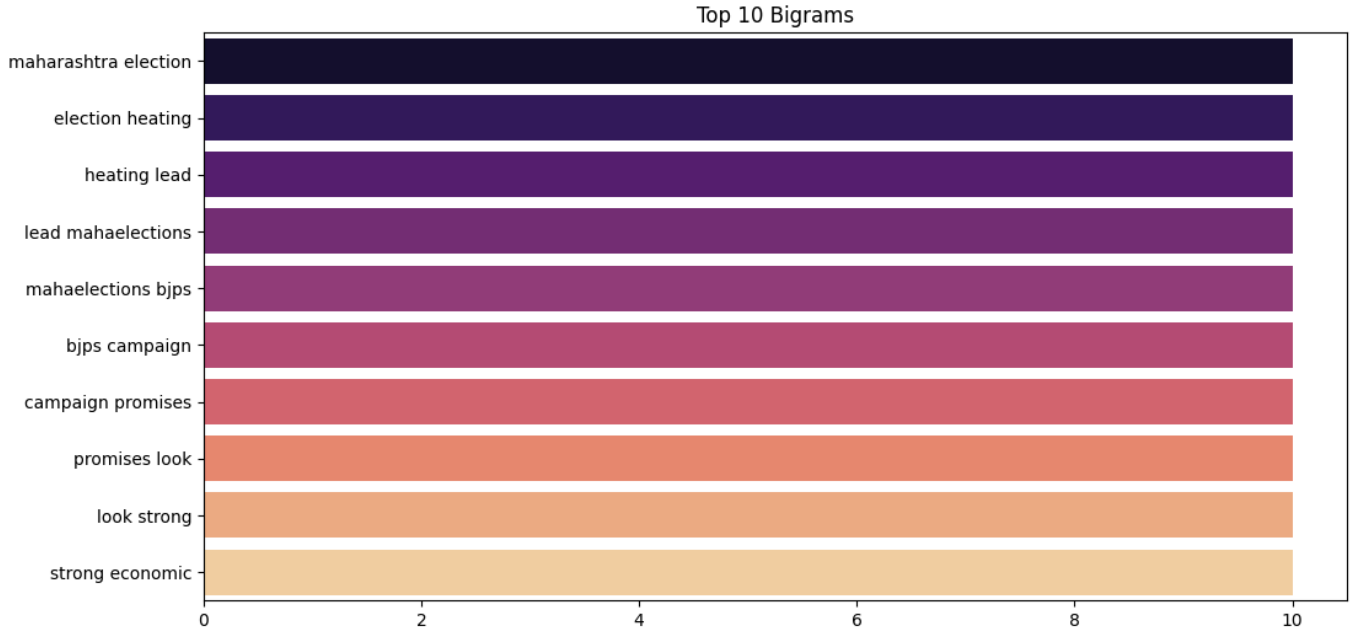
```
# === Cell 1: Imports, Installations, and NLTK Downloads ===
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt
import seaborn as sns
from textblob import TextBlob
from wordcloud import WordCloud
from collections import Counter
import nltk
from nltk.corpus import stopwords
from nltk.util import ngrams

# Install the ntscraper library if not already installed
!pip install ntscraper

from ntscraper import Nitter  # The scraping library

print("Initializing Scraper...")
from ntscraper import Nitter  # Added import statement to ensure Nitter is defined
scraper = Nitter(log_level=1, skip_instance_check=False)

# Define search query and number of tweets
# NOTE: 'terms' can be a hashtag or keywords
search_term = "Maharashtra Assembly Election 2024"
num_tweets = 100

print(f"Scraping {num_tweets} tweets for '{search_term}'... This may take a moment.")

try:
    # Scrape tweets
    scraped_data = scraper.get_tweets(search_term, mode='term', number=num_tweets)

    # Extract relevant data into a list
    final_tweets = []
    # Check if 'tweets' key exists and is not empty before iterating
    if 'tweets' in scraped_data and scraped_data['tweets']:
        for tweet in scraped_data['tweets']:
            final_tweets.append([tweet['date'], tweet['user']['name'], tweet['text']])
        # Create DataFrame only if tweets were successfully scraped
        df = pd.DataFrame(final_tweets, columns=['Date', 'User', 'Tweet'])
        print(f"Successfully scraped {len(df)} tweets.")
    else:
        raise ValueError("No tweets found or scraper returned empty data.")

except Exception as e:
    # FALLBACK DATA (Just in case scraping fails due to network issues or no tweets found)
    data = {
        'Tweet': [
            "Maharashtra election 2024 is heating up! Who will lead? #MahaElections",
```

```
            "BJP's campaign promises look strong for economic growth. #VoteBJP",
            "Congress focusing on farmer welfare and unemployment. #CongressForFarmers",
            "Shiv Sena's local development agenda resonates with many. #ShivSena",
            "NCP emphasizing social justice and education reforms. #NCP",
            "Voter turnout expected to be high this year. Get ready to #VoteMaharashtra",
            "Candidate debates are getting intense. Important to listen to all
perspectives.",
            "Infrastructure development is a key issue in Mumbai. We need better roads!",
            "Rural Maharashtra demands better healthcare facilities. Politicians, take
note!",
            "Youth engagement in this election is unprecedented. New generation, new hope.",
            "Corruption allegations against some candidates are worrying the public.",
            "Digital campaigns are reaching every corner of the state. Technology in
elections!",
            "Opinion polls are showing a tight race. Every vote counts! #Election2024",
            "Local self-governance needs strengthening for grassroots development.",
            "Women's safety and empowerment are crucial topics this election season.",
            "Environmental issues like water scarcity are impacting agricultural regions.",
            "Small businesses are hoping for policies that boost their growth.",
            "Will the incumbent government retain power? The suspense is real!",
            "New faces emerging in several constituencies, bringing fresh ideas.",
            "Don't forget to check your voter ID and polling booth location! #Democracy"
        ] * 10 # Increased to create 200 synthetic tweets
    }
    df = pd.DataFrame(data)


# === Cell 3: Data Cleaning & Preprocessing ===
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    # 3.1 Convert text to Lower Case
    text = str(text).lower()

    # 3.2 Remove the links (URLs)
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)

    # 3.3 Remove anything except the English language and space
    # (Removes numbers, special chars like #, @, emojis)
    text = re.sub(r'[^a-z\s]', '', text)

    # 3.4 Remove Stop words
    tokens = text.split()
    filtered_tokens = [word for word in tokens if word not in stop_words]

    return " ".join(filtered_tokens)

# Apply preprocessing
df['Cleaned_Tweet'] = df['Tweet'].apply(preprocess_text)

# Remove empty rows after cleaning
df = df[df['Cleaned_Tweet'] != '']

print("\nSample of Cleaned Data:")
print(df[['Tweet', 'Cleaned_Tweet']].head())

# === Cell 4: Visualize Top 20 Most Common Words ===
all_words = ' '.join([text for text in df['Cleaned_Tweet']])
tokenized_words = all_words.split()
```

```
word_counts = Counter(tokenized_words)
common_words_df = pd.DataFrame(word_counts.most_common(20), columns=['Word', 'Count'])

plt.figure(figsize=(12, 6))
sns.barplot(x='Count', y='Word', data=common_words_df, palette='viridis')
plt.title('Top 20 Most Common Words')
plt.show()

# === Cell 5: Visualize Top 10 Bigrams ===
bi_grams = ngrams(tokenized_words, 2)
bi_gram_counts = Counter(bi_grams)
top_bigrams = bi_gram_counts.most_common(10)

bigram_labels = [' '.join(bg[0]) for bg in top_bigrams]
bigram_values = [bg[1] for bg in top_bigrams]

plt.figure(figsize=(12, 6))
sns.barplot(x=bigram_values, y=bigram_labels, palette='magma')
plt.title('Top 10 Bigrams')
plt.show()

# === Cell 6: Sentiment Analysis ===
def get_sentiment(text):
    analysis = TextBlob(text)
    if analysis.sentiment.polarity > 0:
        return 'Positive'
    elif analysis.sentiment.polarity < 0:
        return 'Negative'
    else:
        return 'Neutral'

df['Sentiment'] = df['Cleaned_Tweet'].apply(get_sentiment)

print("\nSentiment Distribution:")
print(df['Sentiment'].value_counts())

# === Cell 7: Positive Word Cloud ===
plot_wordcloud('Positive', 'Word Cloud of Positive Words')

# === Cell 8: Negative Word Cloud ===
plot_wordcloud('Negative', 'Word Cloud of Negative Words')

# === Cell 9: Neutral Word Cloud ===
plot_wordcloud('Neutral', 'Word Cloud of Neutral Words')
```

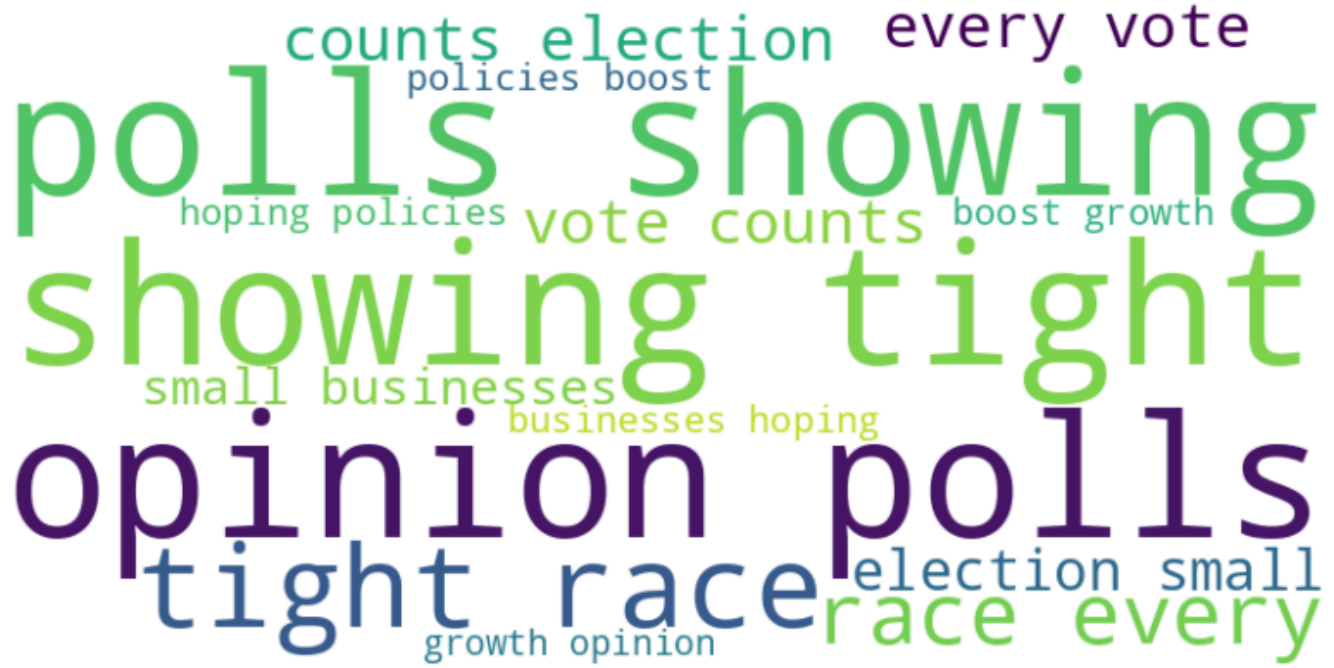
Output:


Top 20 Most Common Words
election
development
new
maharashtra
growth
local
ncp
voter
better
every
heating
lead
mahaelections
bjps
campaign
promises
look
strong
economic
votebjp
Word
0
5
10
15
20
25
30
35
40
Count


Top 10 Bigrams
maharashtra election
election heating
heating lead
lead mahaelections
mahaelections bjps
bjps campaign
campaign promises
promises look
look strong
strong economic
0
2
4
6
8
10

Word Cloud of Positive Words

Word Cloud of Negative Words



Word Cloud of Neutral Words